Human Verification: `Containers.Comonoid`

Directed Containers as Comonoids in $(\mathbf{Cont}, \triangleleft, I)$

Machine-generated from `lean/Comonoid.lean` for review by Neil Ghani

July 2026

Abstract

This document presents every definition and theorem *statement* from the Lean 4 module `Containers.Comonoid` in three parallel forms: the original Lean, a readable Haskell rendering, and precise mathematical English. Proofs are omitted — they are machine-checked by Lean. The purpose is to let a human verifier confirm that the formalisation captures the intended mathematics.

Contents

1 Overview **1**

2 Auxiliary lemmas **2**
2.1 Lemma: `ContainerMorphism.ext_eq` 2
2.2 Lemma: `Container.cast_symm_cast` 2
2.3 Lemma: `Container.seq_pos_transport` 3

3 Comonoid structure **3**
3.1 Structure: `Container.Comonoid` 3

4 The directed container is a comonoid **4**
4.1 Definition: `DirectedContainer.coCounit` 4
4.2 Definition: `DirectedContainer.coComult` 4
4.3 Theorem: `DirectedContainer.toComonoid` 5

5 Translation conventions **5**

1 Overview

The category **Cont** of containers carries a monoidal structure $(\triangleleft, I)$, where the *sequential composition* $C \triangleleft D$ is the container whose extension is $\mathbf{Ext}\, C \circ \mathbf{Ext}\, D$. A **comonoid** in $(\mathbf{Cont}, \triangleleft, I)$ is a container C equipped with:

- a *counit morphism* $\varepsilon : C \to I$, and
- a *comultiplication morphism* $\delta : C \to C \triangleleft C$,

satisfying the two counit laws and coassociativity, all stated as equalities of container morphisms — *without ever passing to* $[\mathbf{Set}, \mathbf{Set}]$.

The main theorem is that every directed container is canonically a $\triangleleft$-comonoid, via `DirectedContainer.toComon`
The comonoid laws correspond exactly to the directed-container axioms D1–D5:

Law	Comonoid	DC axioms
Left counit	$\delta;(\varepsilon \triangleleft C) = \lambda^{-1}$	D1 (shapes) + D2 (positions)
Right counit	$\delta;(C \triangleleft \varepsilon) = \rho^{-1}$	D3
Coassociativity	$\delta;(\delta \triangleleft C);\alpha = \delta;(C \triangleleft \delta)$	D4 (shapes) + D5 (positions)

Everything is **Type**-level, Lean 4 core, no Mathlib.

2 Auxiliary lemmas

2.1 Lemma: `ContainerMorphism.ext_eq`

Proposition 2.1 (Pointwise extensionality for morphisms)**.** Two container morphisms $(f, f_P), (g, g_P) : C \to D$ are equal whenever:

- their shape maps agree pointwise: $\forall s,\ f(s) = g(s)$, and
- their position maps agree after transporting along that pointwise equality: $\forall s, p,\ f_P(s,p) = g_P(s, (h_s)_* p)$, where $h_s : f(s) = g(s)$.

This is stated with a *pointwise* shape equality deliberately, so that the transport h_s appears literally in the position obligation and can be rewritten by `Container.seq_pos_transport`.

Lean.

```
theorem ContainerMorphism.ext_eq {C D : Container} {f g : C.Shape → D
    .Shape}
    {fp : (s : C.Shape) → D.Pos (f s) → C.Pos s}
    {gp : (s : C.Shape) → D.Pos (g s) → C.Pos s}
    (hs : ∀ s, f s = g s) (hp : ∀ s p, fp s p = gp s (hs s cast p)) :
    (⟨f, fp⟩ : ContainerMorphism C D) = ⟨g, gp⟩
```

Haskell.

```
-- Morphism extensionality: two morphisms with the same shape map (
    pointwise)
-- and compatible position maps are equal.  In Haskell this is just
    function
-- extensionality applied to both components.
```

2.2 Lemma: `Container.cast_symm_cast`

Proposition 2.2 (Round-trip transport cancellation)**.** For any type family $P : \alpha \to \mathbf{Type}$, equality $h : a = b$, and element $x : P(a)$,

$$h_*^{-1}(h_* x) \;=\; x.$$

Transporting along h and back is the identity.

Lean.

```
theorem Container.cast_symm_cast {alpha : Type} {P : alpha → Type}
    {a b : alpha} (h : a = b) (x : P a) :
    (h.symm cast (h cast x : P b) : P a) = x
```

Haskell.

```
-- The round-trip  coerce . coerce = id  holds by parametricity / type
    safety.
-- In Lean this requires an explicit proof because transport is not
    definitionally
-- involutive.
```

2.3 Lemma: `Container.seq_pos_transport`

Proposition 2.3 (Transport on positions of sequential composition)**.** Let C, E be containers, $t : E.\mathrm{Shape}$, and $f, g : E.\mathrm{Pos}(t) \to C.\mathrm{Shape}$ inner-shape assignments with $h : \forall u,\ f(u) = g(u)$. Then transporting a position $\langle u, q\rangle : (E \triangleleft C).\mathrm{Pos}\langle t, f\rangle$ along the induced equality acts componentwise:

$$(h_*)_*\langle u, q\rangle \;=\; \langle u,\ (h_u)_* q\rangle \;\in\; (E \triangleleft C).\mathrm{Pos}\langle t, g\rangle.$$

That is, the outer position u is unchanged, and only the inner position q is transported along $h(u) : f(u) = g(u)$.

Lean.

```
theorem Container.seq_pos_transport {C E : Container} {t : E.Shape}
    {f g : E.Pos t → C.Shape} (h : ∀ u, f u = g u)
    (u : E.Pos t) (q : C.Pos (f u)) :
    (congrArg (Sigma.mk (beta := fun k ⇒ E.Pos k → C.Shape) t) (
       funext h) cast
        (⟨u, q⟩ : (E seq C).Pos ⟨t, f⟩) : (E seq C).Pos ⟨t, g⟩)
      = (⟨u, (h u cast q : C.Pos (g u))⟩ : (E seq C).Pos ⟨t, g⟩)
```

Haskell.

```
-- The position type of (E seq C) over shape (t, f) is  (u : E.Pos t,
   C.Pos (f u)).
-- Transporting along a pointwise equality h : forall u. f u = g u
-- leaves u unchanged and coerces the inner position via h u.
-- No Haskell encoding is natural; this is purely a transport-
   bookkeeping lemma.
```

Fidelity note. This lemma and `ContainerMorphism.ext_eq` are the two main tools for managing dependent transports in this file. They have no non-trivial Haskell analogues, since Haskell lacks dependent types. Their role is to isolate and discharge all "bureaucratic" casts so that the mathematical content of the comonoid laws is visible in the main proof of `toComonoid`.

3 Comonoid structure

3.1 Structure: `Container.Comonoid`

Definition 3.1 (Comonoid in $(\mathbf{Cont}, \triangleleft, I)$)**.** A **comonoid** on a container C consists of:

- a counit morphism $\varepsilon : C \to I$,
- a comultiplication morphism $\delta : C \to C \triangleleft C$,

satisfying:

- **Left counit:** $\delta\,;(\varepsilon \triangleleft \mathrm{id}_C) = \lambda_C^{-1}$,
- **Right counit:** $\delta\,;(\mathrm{id}_C \triangleleft \varepsilon) = \rho_C^{-1}$,
- **Coassociativity:** $(\delta\,;(\delta \triangleleft \mathrm{id}_C))\,;\alpha_{C,C,C} = \delta\,;(\mathrm{id}_C \triangleleft \delta)$.

Here λ, ρ, α are the left unitor, right unitor, and associator of the monoidal category $(\mathbf{Cont}, \triangleleft, I)$, and ; is morphism composition in **Cont**.

Lean.

```
structure Container.Comonoid (C : Container) where
  counit : ContainerMorphism C Container.I
  comult : ContainerMorphism C (C seq C)
  left_counit :
    comult.comp (Container.seq2 counit (ContainerMorphism.id C))
```

```
      = (Container.leftUnitor C).inv
  right_counit :
    comult.comp (Container.seq2 (ContainerMorphism.id C) counit)
      = (Container.rightUnitor C).inv
  coassoc :
    (comult.comp (Container.seq2 comult (ContainerMorphism.id C))).
       comp
        (Container.associator C C C).hom
      = comult.comp (Container.seq2 (ContainerMorphism.id C) comult)
```

Haskell.

```
-- A comonoid in (Cont, seq, I):
-- class Comonoid c where
--   counit :: c → I          -- shape: always (); pos: root
--   comult :: c → c seq c     -- shape: (s, sub s); pos: shift s p q
-- Laws (stated as equations between natural transformations):
--   left_counit  : comult ; (counit seq id) = leftUnitor_inv
--   right_counit : comult ; (id seq counit) = rightUnitor_inv
--   coassoc      : (comult ; (comult seq id)) ; assoc = comult ; (id
   seq comult)
```

Fidelity note. The laws are stated *internally* in **Cont** — as equalities of container morphisms, not as equalities of natural transformations. This is the key distinction from the comonad statement in `Containers.Directed`: there the laws mention **Ext**, here they do not. Unfolding the definitions shows the two formulations are equivalent, and the proof of `toComonoid` makes this explicit.

4 The directed container is a comonoid

4.1 Definition: `DirectedContainer.coCounit`

Definition 4.1 (Counit morphism)**.** The **counit container morphism** $\varepsilon : C \to I$ of a directed container D sends every shape to $()$ and pulls back positions to the root:

$$\varepsilon.\mathrm{onShapes}(s) \;=\; (), \qquad \varepsilon.\mathrm{onPos}(s, _) \;=\; o(s).$$

Lean.

```
def coCounit (D : DirectedContainer) : ContainerMorphism D.toContainer
    Container.I where
  onShapes := fun _ ⇒ ()
  onPos := fun s _ ⇒ D.root s
```

Haskell.

```
coCounit :: DirectedContainer s p → Morph s p () ()
coCounit d = Morph { onShapes = \_ → (), onPos = \s _ → root d s }
```

4.2 Definition: `DirectedContainer.coComult`

Definition 4.2 (Comultiplication morphism)**.** The **comultiplication container morphism** $\delta : C \to C \triangleleft C$ of a directed container D is:

$$\delta.\mathrm{onShapes}(s) \;=\; \langle s, s \downarrow - \rangle, \qquad \delta.\mathrm{onPos}(s, \langle p, q \rangle) \;=\; p \oplus q.$$

The shape map sends s to the pair $(s, s \downarrow -)$; the position map combines a pair $\langle p, q \rangle$ of positions via shift.

Lean.

```
def coComult (D : DirectedContainer) :
    ContainerMorphism D.toContainer (D.toContainer seq D.toContainer)
        where
  onShapes := fun s ⇒ ⟨s, D.sub s⟩
  onPos := fun s pq ⇒ D.shift s pq.1 pq.2
```

Haskell.

```
coComult :: DirectedContainer s p → Morph s p (s, s → p, p → s) (p,
    p) p
-- Shape map:  s |→ (s, sub d s)
-- Pos map:    (s, (p, q)) |→ shift d s p q
```

4.3 Theorem: `DirectedContainer.toComonoid`

Theorem 4.3 (Every directed container is a $\triangleleft$-comonoid)**.** Let D be a directed container. Then the pair $(\texttt{coCounit}(D), \texttt{coComult}(D))$ makes D.toContainer into a comonoid in $(\mathbf{Cont}, \triangleleft, I)$. Specifically:

- **Left counit**: follows from D1 (shapes) and D2 (positions), using `Container.seq_pos_transport` to handle the dependent cast.
- **Right counit**: follows immediately from D3.
- **Coassociativity**: follows from D4 (shapes) and D5 (positions), using two applications of `Container.seq_pos_transport` together with `Container.cast_symm_cast` to cancel the D4 round-trip.

Lean.

```
def toComonoid (D : DirectedContainer) : Container.Comonoid D.
   toContainer where
  counit := D.coCounit
  comult := D.coComult
  left_counit := ...   -- proof omitted
  right_counit := ...  -- proof omitted
  coassoc      := ...  -- proof omitted
```

Haskell.

```
-- Every directed container (s, p, root, sub, shift) satisfying D1--D5
-- gives a comonoid in (Cont, seq, I):
--   counit = Morph (\_ → ()) (\s _ → root s)
--   comult = Morph (\s → (s, sub s)) (\s (p,q) → shift s p q)
-- and the comonoid laws follow from D1--D5.
```

Fidelity note. The Lean definition is a `def`, not a `theorem`, because it constructs a term of type `Container.Comonoid D.toContainer`. The proof fields (`left_counit`, `right_counit`, `coassoc`) are filled by tactic blocks; these are omitted here.

The correspondence between comonoid laws and DC axioms is exact:

$$\text{right counit} \leftrightarrow \text{D3}, \quad \text{left counit} \leftrightarrow \text{D1} + \text{D2}, \quad \text{coassociativity} \leftrightarrow \text{D4} + \text{D5}.$$

The dependent casts arising from D1 and D4 (which are only propositional shape equalities) are managed by the two auxiliary lemmas proved in Section **??** above.

5 Translation conventions

1. **Sequential composition $\triangleleft$.** The operator `<|` (written `seq` in listings) in Lean is the sequential composition of containers. Its extension satisfies $\mathbf{Ext}(C \triangleleft D) \cong \mathbf{Ext}\, C \circ \mathbf{Ext}\, D$.

In Haskell it would be a type-level composition of functors.

2. **Unit container I.** The unit I has a single shape () and no positions. Its extension is the identity functor.

3. **Monoidal structure.** The left unitor λ, right unitor ρ, and associator α are container isomorphisms (pairs of inverse container morphisms) proved in `Containers.Monoidal`.

4. **Laws stated internally.** The comonoid laws are equations between *container morphisms*, not between natural transformations. This is significant: the theorem "directed containers are exactly comonoids in $(\mathbf{Cont}, \triangleleft, I)$" holds at the internal level, sidestepping the functor category $[\mathbf{Set}, \mathbf{Set}]$ entirely.

5. **Proof bodies omitted.** Only definitions and theorem *statements* are translated. Lean checks the proofs; the human verifier checks that the *statements* match the intended mathematics.